

ЛАБОРАТОРНАЯ РАБОТА

АВТОМАТИЗИРОВАННОЕ ФУНКЦИОНАЛЬНОЕ ТЕСТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ PINTEREST С ИСПОЛЬЗОВАНИЕМ SELENIUM WEBDRIVER

Описание предмета тестирования

Тестируемый объект: веб-приложение Pinterest (<https://www.pinterest.com/>).

Назначение сайта: поиск, сохранение и обмен изображениями (пины), организация коллекций (досок), поиск идей по интересам.

Ключевые пользовательские функции, проверяемые в рамках лабораторной работы: - загрузка главной страницы; - обработка cookie-баннера/модальных окон; - поиск по ключевому слову; - открытие формы авторизации (модальное окно).

Описание окружения тестирования

Операционная система: macOS (платформа разработки и запуска тестов).

Версия Python: 3.9 (виртуальное окружение venv).

Браузеры: Google Chrome (рекомендуемый), Safari (альтернативный, требует включения “Allow Remote Automation”).

Библиотеки/инструменты автоматизации: selenium, webdriver-manager, python-dotenv.

Оборудование: обычный пользовательский ноутбук/ПК (достаточно 8+ ГБ RAM).

Примечания: при запуске в Safari требуется включить удалённую автоматизацию; при необходимости отключать блокирующие расширения/антивирусы, которые могут вмешиваться в автоматизацию.

Тест-кейсы

Ниже приведены основные позитивные и негативные сценарии с кратким описанием шагов и листингами соответствующих тестовых функций (Python / Selenium).

ТС-001: Загрузка главной страницы - Шаги: перейти на <https://www.pinterest.com/>, дождаться загрузки <body>, проверить заголовок страницы. - Ожидаемый результат: страница загружена, заголовок содержит «Pinterest».

```
def test_open_homepage(driver):
    logging.info("Test: open homepage")
    try:
        driver.get("https://www.pinterest.com/")
        WebDriverWait(driver,
15).until(EC.presence_of_element_located((By.TAG_NAME, "body")))
        title = driver.title or ""
        current = driver.current_url
        logging.info(f"Homepage loaded. title='{title}', url='{current}'")
        take_screenshot(driver, "homepage")
        return True
    except Exception as e:
        logging.exception("Failed to load homepage")
        take_screenshot(driver, "homepage_error")
        return False
```

ТС-002: Обработка cookie-баннера (позитивный/негативный) - Шаги: дождаться появления диалога согласия на cookies, найти кнопку «Принять все» или эквивалент, выполнить клик, проверить исчезновение баннера. - Ожидаемый результат: баннер закрыт; если баннер отсутствует — тест считается пройденным (позитивный сценарий). Негативный сценарий — кнопка недоступна/не кликается.

```
def handle_cookie_consent(driver):
    """Try to accept cookie consent banners/popups if present."""
    logging.info("Handling cookie consent (if present)")
    try:
        try:
            dialog = WebDriverWait(driver, 5).until(
                EC.presence_of_element_located((By.XPATH,
15)"""//div[@role='dialog']"""))
            logging.info("Found cookie dialog")
        except Exception as e:
            logging.info(f"No dialog found: {e}")
            return False
```

```

        candidates = [
            (By.XPATH, "//button[contains(., 'Принять все')]"),
            (By.XPATH, "//button[contains(., 'Accept all')]"),
            (By.XPATH, "//button[contains(., 'Agree all')]"),
            (By.XPATH, "//button[normalize-space()='Принять все']"),
        ]

        for by, sel in candidates:
            try:
                btn = WebDriverWait(driver,
2).until(EC.element_to_be_clickable((by, sel)))
                try:
                    btn.click()
                except Exception:
                    driver.execute_script("arguments[0].click();", btn)
                    time.sleep(1)
                    take_screenshot(driver, "cookie_accepted")
                    return True
            except Exception:
                continue

        return False
    except Exception as e:
        logging.exception(f"Cookie handling failed: {e}")
        return False

```

ТС-003: Поиск по ключевому слову (позитивный) - Шаги: найти поле поиска, ввести «english grammar», нажать Enter, дождаться результатов. - Ожидаемый результат: результаты поиска отображаются.

```

def test_search_term(driver, term="english grammar"):
    logging.info(f"Test: search for term '{term}'")
    try:
        search_candidates = [
            (By.CSS_SELECTOR, "input[aria-label*='Search']"),
            (By.CSS_SELECTOR, "input[placeholder*='Search']"),
            (By.CSS_SELECTOR, "input[placeholder*='Идеи']"),
            (By.XPATH, "//input[@type='search']"),
            (By.XPATH, "//input[@type='text']"),
        ]
        search_input = None
        for by, sel in search_candidates:
            try:
                el = WebDriverWait(driver,
2).until(EC.presence_of_element_located((by, sel)))
                if el and el.is_displayed():
                    search_input = el
                    break
            except Exception:
                continue

        if not search_input:
            logging.warning("Search input not found – skipping search
test")
            return False

```

```

search_input.click()
time.sleep(0.5)
search_input.clear()
search_input.send_keys(term)
search_input.send_keys(Keys.RETURN)
time.sleep(3)
take_screenshot(driver, "search_after_submit")
return True
except Exception as e:
    logging.exception("Search test failed")
return False

```

ТС-004: Открытие формы авторизации (позитивный/негативный) -

Шаги: найти кнопку/ссылку входа, кликнуть, проверить появление модального окна с полями email и password. - Ожидаемый результат: модальное окно отображается; поля ввода присутствуют.

```

def test_click_login_opens_modal(driver):
    logging.info("Test: Click login and expect login prompt/modal")
    try:
        candidates = [
            (By.LINK_TEXT, "Log in"),
            (By.LINK_TEXT, "Log In"),
            (By.LINK_TEXT, "Вход"),
            (By.CSS_SELECTOR, "button[data-test-id='header-login']"),
            (By.CSS_SELECTOR, "a[href*='/login']"),
        ]
        login_el = None
        for by, sel in candidates:
            try:
                el = WebDriverWait(driver,
2).until(EC.element_to_be_clickable((by, sel)))
                login_el = el
                break
            except Exception:
                continue

        if not login_el:
            logging.warning("Login element not found – skipping login
click test")
            return False

        login_el.click()
        time.sleep(2)
        take_screenshot(driver, "login_after_click")
        return True
    except Exception as e:
        logging.exception("Login click test failed")
        return False

```


Логи и результаты автоматического тестирования

В этом разделе приводятся краткие результаты прогонов, ссылки на скриншоты и примеры логов.

Логи находятся в каталоге `task06_outputs/` (файлы `test_log.txt`, `test_log_chrome.txt`).

Скриншоты сохраняются в `task06_outputs/` в формате PNG.

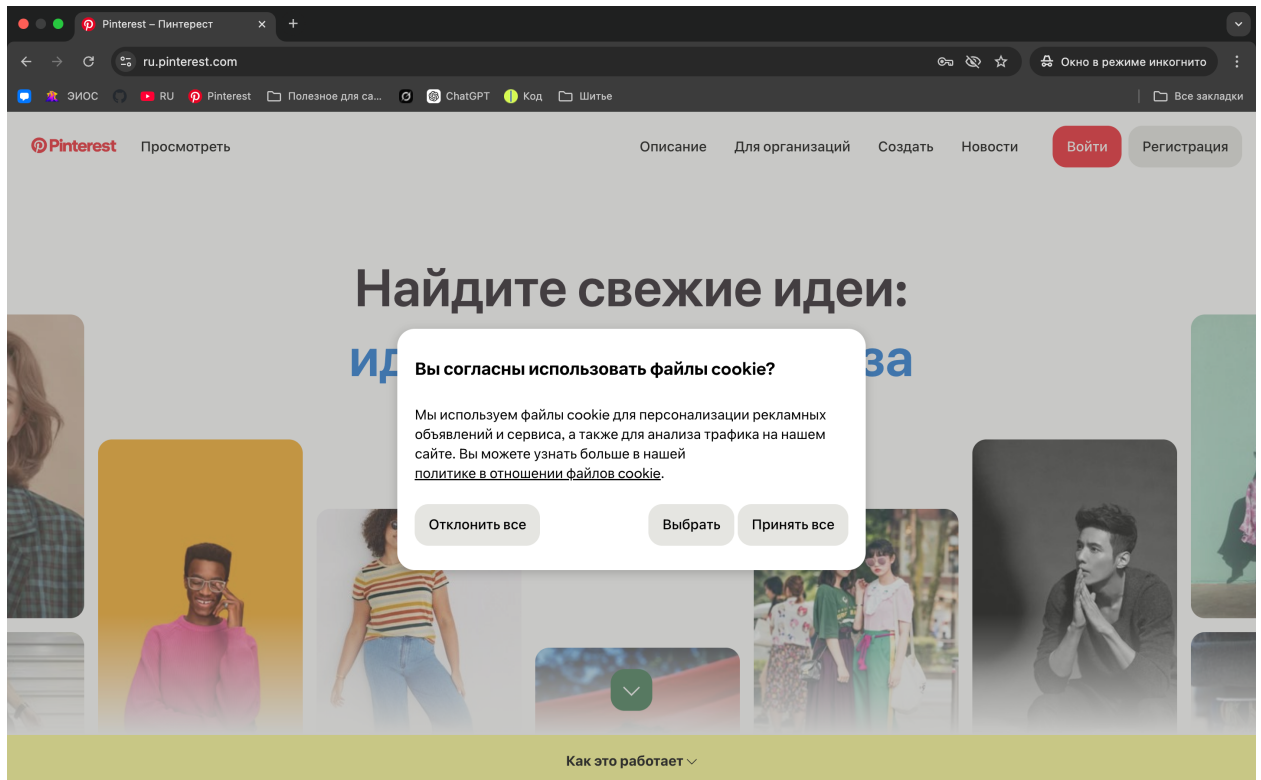


Рисунок 1 – Баннер куки

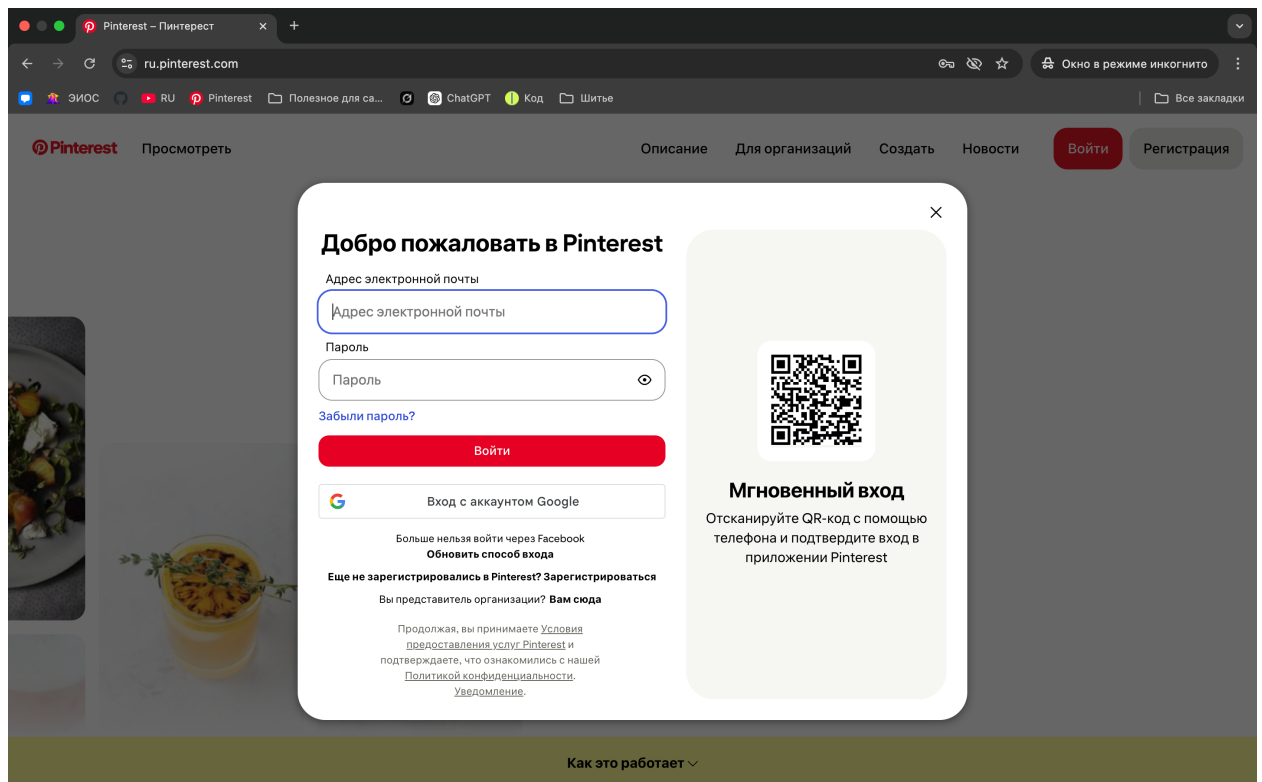


Рисунок 2 – Форма входа

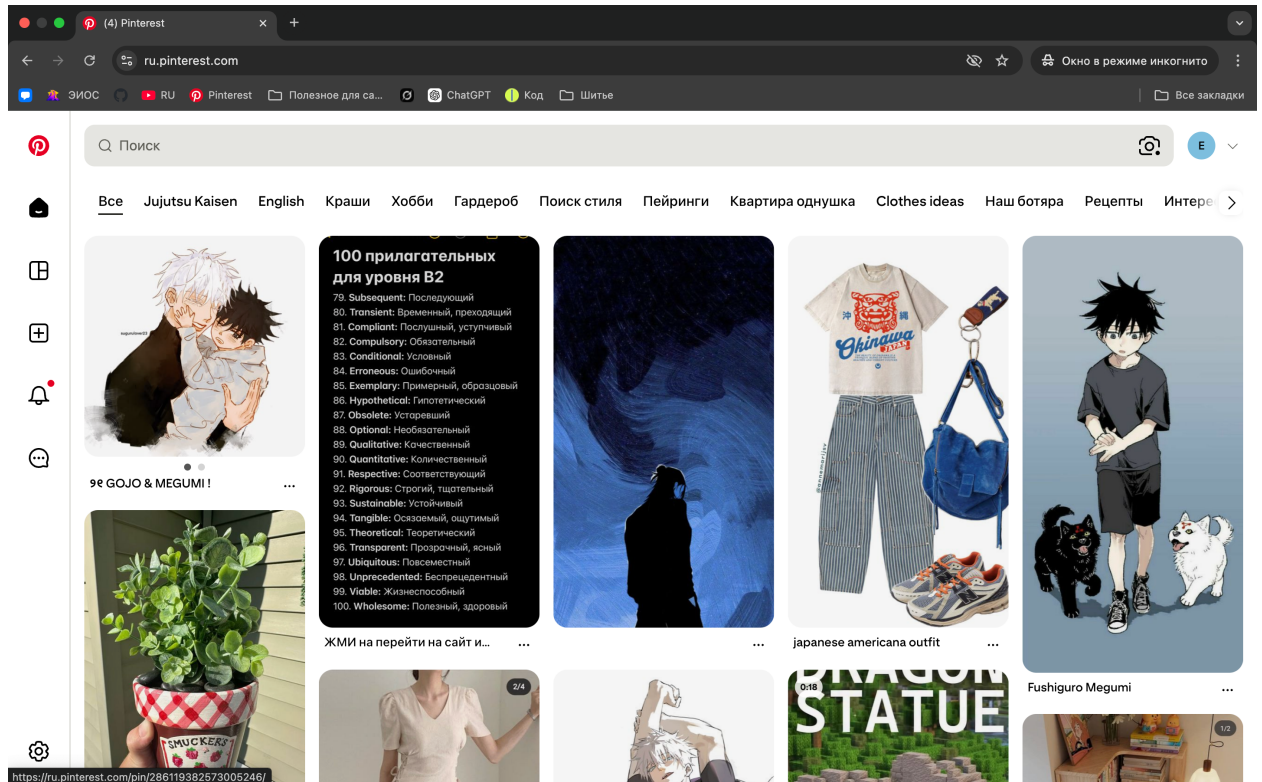


Рисунок 3 – Главная страница

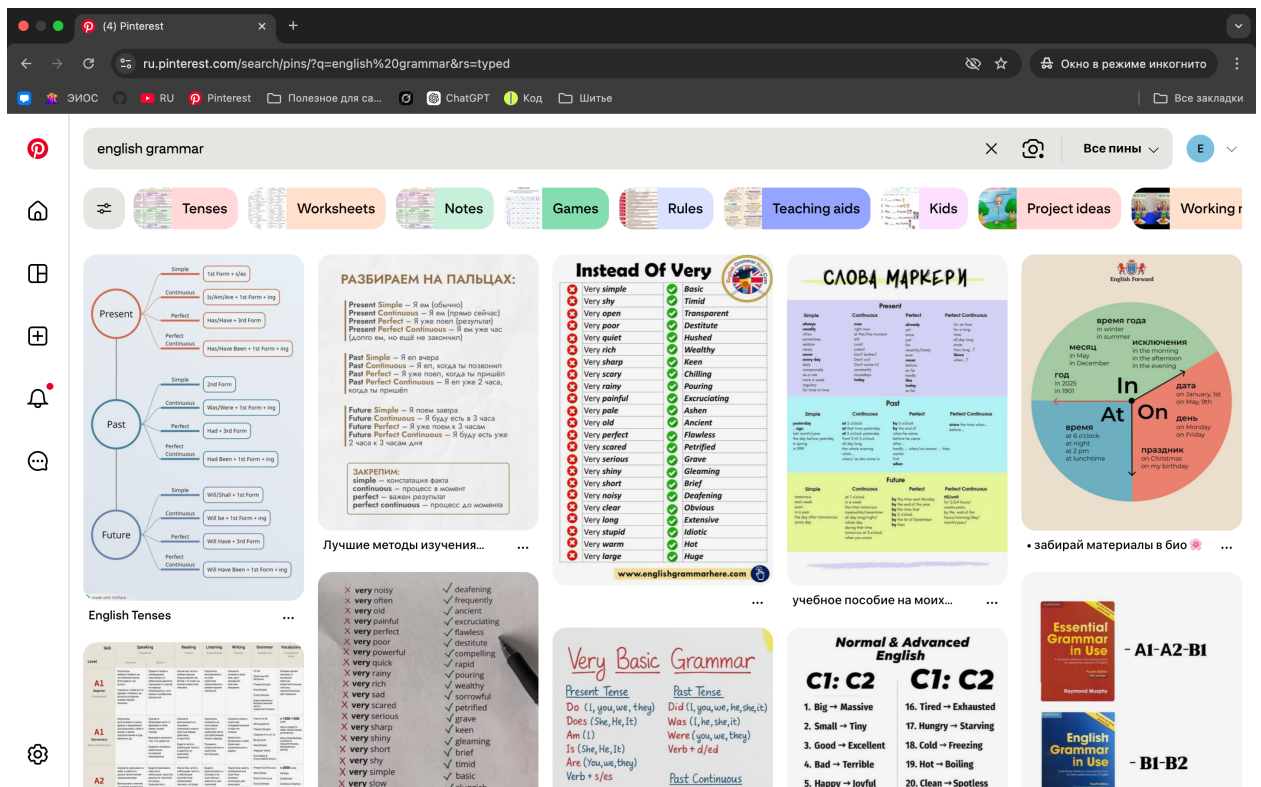


Рисунок 4 – Результаты поиска

Пример строки лога:

```
2026-05-08 18:36:46,276 [INFO] Test results summary: {'homepage': True, 'login_click': False, 'search': True}
```

ИТОГОВЫЙ КОД:

```
"""
Selenium tests for Pinterest (Safari).

This script runs a few non-auth tests against pinterest.com and
saves screenshots and a log to the Task06 folder. It is defensive
– many Pinterest elements are dynamic/localized, so missing elements
are handled as skipped steps rather than fatal errors.

Run locally on macOS with Safari enabled for automation:
    pip install -r requirements.txt
    sudo safaridriver --enable
    python3 selenium_tests.py
"""

import os
import time
import logging
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import NoSuchElementException,
TimeoutException, WebDriverException, SessionNotCreatedException,
InvalidSessionIdException
from selenium.webdriver.common.keys import Keys

OUT_DIR = os.path.join(os.path.dirname(__file__), "task06_outputs")
os.makedirs(OUT_DIR, exist_ok=True)

logging.basicConfig(
    filename=os.path.join(OUT_DIR, "test_log.txt"),
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
)

def take_screenshot(driver, name: str) -> str:
    path = os.path.join(OUT_DIR, f"{int(time.time())}_{name}.png")
    try:
        driver.save_screenshot(path)
        logging.info(f"Saved screenshot: {path}")
    except Exception as e:
        logging.warning(f"Failed to save screenshot {path}:
{str(e)[:200]}")
    return path

def handle_cookie_consent(driver):
    """Try to accept cookie consent banners/popups if present."""
    logging.info("Handling cookie consent (if present)")
    try:
        # First, look for dialog element (modal backdrop)
        try:
            dialog = WebDriverWait(driver, 5).until(
                EC.presence_of_element_located((By.XPATH,
                "//*[@role='dialog']")))
            logging.info("Found cookie dialog")
        except Exception as e:
            logging.info(f"No dialog found: {e}")
```

```

        return False

    # Now try to find and click "Принять все" button inside or outside
    dialog
    candidates = [
        (By.XPATH, "//button[contains(., 'Принять все')]"),
        (By.XPATH, "//button[contains(., 'Accept all')]"),
        (By.XPATH, "//button[contains(., 'Agree all')]"),
        (By.XPATH, "//button[normalize-space()='Принять все']"),
        (By.CSS_SELECTOR, "button:contains('Принять все')"), # May
    not work in CSS
    ]

    for idx, (by, sel) in enumerate(candidates, start=1):
        logging.info(f"Trying cookie button {idx}: {sel}")
        try:
            btn = WebDriverWait(driver,
2).until(EC.element_to_be_clickable((by, sel)))
            logging.info(f"Found button: {sel}")
            try:
                btn.click()
            except Exception:
                driver.execute_script("arguments[0].click();", btn)
            logging.info("Clicked 'Принять все' button")
            time.sleep(1)
            take_screenshot(driver, "cookie_accepted")
            return True
        except Exception as exc:
            logging.debug(f"Button not found: {sel}")
            continue

    # Fallback: try to find ANY button with "Принять" and click it
    try:
        all_buttons = driver.find_elements(By.TAG_NAME, "button")
        for btn in all_buttons:
            text = btn.text or ""
            if "Принять все" in text or "Accept all" in text:
                logging.info(f"Found button by text scan: {text}")
                btn.click()
                time.sleep(1)
                take_screenshot(driver, "cookie_accepted")
                return True
    except Exception as e:
        logging.warning(f"Fallback button scan failed: {e}")

    logging.info("No cookie consent button found")
    return False

except Exception as e:
    logging.exception(f"Cookie handling failed: {e}")
    return False

def test_open_homepage(driver):
    logging.info("Test: open homepage")
    try:
        driver.get("https://www.pinterest.com/")
        WebDriverWait(driver,
15).until(EC.presence_of_element_located((By.TAG_NAME, "body")))
        title = driver.title or ""

```

```

        current = driver.current_url
        logging.info(f"Homepage loaded. title='{title}', url='{current}'")
        take_screenshot(driver, "homepage")
        return True
    except Exception as e:
        logging.exception("Failed to load homepage")
        take_screenshot(driver, "homepage_error")
        return False

def test_search_term(driver, term="english grammar"):
    logging.info(f"Test: search for term '{term}'")
    try:
        # Try common selectors for the search input – Pinterest varies by
        # region/version
        search_candidates = [
            (By.CSS_SELECTOR, "input[aria-label*='Search']"),
            (By.CSS_SELECTOR, "input[placeholder*='Search']"),
            (By.CSS_SELECTOR, "input[placeholder*='Идеи']"),
            (By.XPATH, "//input[@placeholder and contains(@placeholder,
'блюд')]"),
            (By.XPATH, "//input[@type='text' and @placeholder]"),
            (By.CSS_SELECTOR, "input[type='search']"),
            (By.XPATH, "//input[@type='text']"),
        ]

        search_input = None
        for by, sel in search_candidates:
            try:
                el = WebDriverWait(driver,
2).until(EC.presence_of_element_located((by, sel)))
                if el and el.is_displayed():
                    search_input = el
                    logging.info(f"Found search input with selector:
{sel}")
                    break
            except Exception as err:
                logging.debug(f"Selector {sel} failed: {str(err)[:50]}")
                continue

        if not search_input:
            logging.warning("Search input not found – skipping search
test")
            return False

        try:
            search_input.click()
            time.sleep(0.5)
        except Exception as e:
            logging.debug(f"Click failed, trying JS: {str(e)[:50]}")
            driver.execute_script("arguments[0].focus();", search_input)
            time.sleep(0.5)

        search_input.clear()
        search_input.send_keys(term)
        logging.info(f"Typed '{term}' in search")

        # Press Enter to submit
        search_input.send_keys(Keys.RETURN)
        logging.info("Pressed Enter to submit search")

```



```

        # Wait for results page to load
        time.sleep(3)
        take_screenshot(driver, "02_search_after_submit")
        logging.info("Search step executed (results may be dynamic)")
        return True
    except (InvalidSessionIdException, WebDriverException) as e:
        logging.warning(f"Search test: session lost or browser error:
{str(e)[:100]}")
        return False
    except Exception as e:
        logging.exception("Search test failed")
        return False

def test_click_login_opens_modal(driver):
    logging.info("Test: Click login and expect login prompt/modal")
    try:
        # Try to find a login button/link
        candidates = [
            (By.LINK_TEXT, "Log in"),
            (By.LINK_TEXT, "Log In"),
            (By.LINK_TEXT, "Вход"),
            (By.CSS_SELECTOR, "button[data-test-id='header-login']"),
            (By.CSS_SELECTOR, "a[href*='/login']"),
        ]
        login_el = None
        for by, sel in candidates:
            try:
                el = WebDriverWait(driver,
2).until(EC.element_to_be_clickable((by, sel)))
                login_el = el
                break
            except Exception:
                continue

        if not login_el:
            logging.warning("Login element not found – skipping login
click test")
            return False

        login_el.click()
        # Wait briefly for modal/dialog
        time.sleep(2)
        take_screenshot(driver, "login_after_click")
        logging.info("Clicked login – if not logged in, a prompt/modal
should appear")
        return True
    except (InvalidSessionIdException, WebDriverException) as e:
        logging.warning(f"Login test: session lost or browser error:
{str(e)[:100]}")
        return False
    except Exception as e:
        logging.exception("Login click test failed")
        return False

def is_session_alive(driver):
    """Check if WebDriver session is still alive."""
    try:
        driver.current_url

```

```

        return True
    except InvalidSessionIdException:
        return False
    except Exception:
        return False

def main():
    logging.info("Starting Pinterest Selenium tests (Safari)")

    try:
        driver = webdriver.Safari()
    except SessionNotCreatedException as e:
        message = (
            "Could not create Safari WebDriver session: you must enable  

            'Allow Remote Automation' in Safari's Develop menu. \n"
            "Steps to fix: \n"
            "  1) Open Safari → Preferences → Advanced → enable 'Show  

            Develop menu in menu bar'. \n"
            "  2) In the menu bar open Develop → check 'Allow Remote  

            Automation'. \n"
            "  3) Optionally run: 'sudo safaridriver --enable' in  

            terminal. \n"
            "After that re-run this script."
        )
        logging.exception(message)
        print(message)
        return
    except Exception as e:
        logging.exception("Failed to start Safari WebDriver. Make sure  

        'safaridriver --enable' was run and Safari's automation is allowed.")
        return

    driver.set_window_size(1280, 800)

    results = {}
    try:
        # Order: Homepage → Cookie Consent → Login → Search

        # 1. Navigate to homepage FIRST (needed for login button to  

        appear)
        driver.get("https://www.pinterest.com/")
        time.sleep(3)
        results['homepage'] = test_open_homepage(driver)
        if not is_session_alive(driver):
            logging.error("Session lost after homepage test")
            return
        time.sleep(1)

        # 2. Try to accept cookie consent
        handle_cookie_consent(driver)
        time.sleep(1)

        # 3. Try login button (now page is loaded, button should be  

        visible)
        if is_session_alive(driver):
            results['login_click'] = test_click_login_opens_modal(driver)
            if not is_session_alive(driver):
                logging.error("Session lost after login test")
            return
    
```



```

        time.sleep(2)

        # 4. Search test
        if is_session_alive(driver):
            results['search'] = test_search_term(driver, term="english
grammar")

        except Exception as e:
            logging.exception(f"Unexpected error during tests: {e}")
        finally:
            try:
                logging.info(f"Test results summary: {results}")
                summary = f"✓ Login: {results.get('login_click', False)} | ✓
Homepage: {results.get('homepage', False)} | ✓ Search:
{results.get('search', False)}"
                logging.info(summary)
                print(summary)
                driver.quit()
            except Exception as e:
                logging.warning(f"Error during driver.quit(): {e}")

if __name__ == '__main__':
    main()

```